Legacy ETL Compiler

Mark London

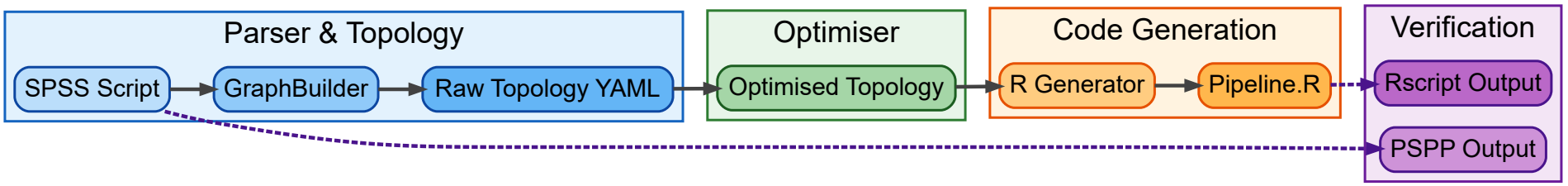
2026-03-05

Overview

- Purpose: migrate SPSS logic into R pipelines
- Verification & validation cycle of output

  This is a proof of concept to explore the value potential of developing it into full product.

Architecture



- PSPP is a free Linux version of SPSS. Whilst not identical, it accepts most SPSS syntax.

What is the legacy-etl-compiler system?

- Takes an existing SPSS pipeline and converts it into another language (e.g., R).
- Uses an intermediate representation to make the compiler deterministic, testable, and optimisable.
- The key intermediate representation is a State Machine built from the original script.

Why not translate directly to R?

- The Syntax Trap: Direct translation maps SPSS syntax directly to R syntax. This fails because the two languages have fundamentally different execution models.
- Hidden Side Effects: SPSS is highly procedural and relies on a mutable “active dataset.” R leans towards functional dataframe manipulation.
- Brittle Output: Without an intermediate logical layer, the resulting R code is often unreadable, unoptimised, and extremely difficult to test for true equivalence.

Why via a State Machine?

- A State Machine models the true underlying *territory* of the data lifecycle, independent of the *map* (the syntax) of either language.
- It makes dependencies explicitly clear and eliminates hidden side effects.
- It safely allows the compiler to reorder, optimise, and mathematically verify transformations while ensuring behavioural equivalence.

What is a State Machine?

- A mathematical model of a program where each operation explicitly transitions the system to a new, immutable state.
- Instead of mutating variables, we create new versions at each step (e.g., `X1`, `X2`, `X3`).
- Each step is deterministic, making complex, messy legacy pipelines significantly easier to reason about.

Order Matters

Side-by-Side Comparison

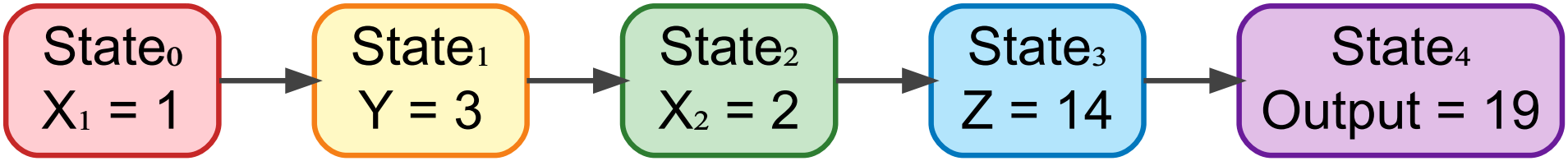
Original Order

```
X <- 1        → X = 1
Y <- X * 3    → Y = 3
X <- X + 1    → X = 2
Z <- X * 7    → Z = 14
sum(X, Y, Z)
Result = 19
```

Modified Order

```
X <- 1        → X = 1
Z <- X * 7    → Z = 7
X <- X + 1    → X = 2
Y <- X * 3    → Y = 6
sum(X, Y, Z)
Result = 15
```

State Machine View (Same Logic)



00_original_spss.txt (Source)

```
* Hello World Pipeline.

* 1. LOAD .
GET DATA
  /TYPE=TXT
  /FILE='hello.csv'
  /ARRANGEMENT=DELIMITED
  /DELIMITERS=','
  /FIRSTCASE=2
  /VARIABLES=
    id F8.0
    name A10.

* 2. SORT (The Test Case).
SORT CASES BY id.

* 3. SAVE (Using TRANSLATE to get a readable CSV).
SAVE TRANSLATE
  /OUTFILE='hello_sorted.csv'
  /TYPE=CSV
  /REPLACE.
```

01_source_verification.txt (PSPP run)

```
Command: pspp hello.sps
Status: ✅ Success

=== OUTPUT ===
```

02_raw_topology.yaml (Compiled Topology)

```
# Pipeline Topology: 3 Operations
----------------------------------------
Operation: op_001_load
  Type:    LOAD_CSV
  Inputs:  []
  Outputs: ['source_hello.csv']
  Params:  {'filename': 'hello.csv', 'format': 'TXT', 'schema': ['id', 'name'], 'skip_rows': 1}

Operation: op_002_sort
  Type:    SORT_ROWS
  Inputs:  ['source_hello.csv']
  Outputs: ['ds_001_sorted']
  Params:  {'keys': 'id', 'order': 'ascending'}

Operation: op_003_save
  Type:    SAVE_BINARY
  Inputs:  ['ds_001_sorted']
  Outputs: ['file_hello_sorted.csv']
  Params:  {'filename': 'hello_sorted.csv'}
```

03_optimized_topology.yaml (Optimized Topology)

```
# Pipeline Topology: 3 Operations
---------------------------------------
Operation: op_001_load
  Type:    LOAD_CSV
  Inputs:  []
  Outputs: ['source_hello.csv']
  Params:  {'filename': 'hello.csv', 'format': 'TXT', 'schema': ['id', 'name'], 'skip_rows': 1}

Operation: op_002_sort
  Type:    SORT_ROWS
  Inputs:  ['source_hello.csv']
  Outputs: ['ds_001_sorted']
  Params:  {'keys': 'id', 'order': 'ascending'}

Operation: op_003_save
  Type:    SAVE_BINARY
  Inputs:  ['ds_001_sorted']
  Outputs: ['file_hello_sorted.csv']
  Params:  {'filename': 'hello_sorted.csv'}
```

04_generated_code.R (Generated R)

```r
# Auto-Generated ETL Script
# Generated by: V&V Compiler
# Dialect: Tidyverse

library(tidyverse)
library(lubridate)
library(haven)

# Op: op_001_load
source_hello.csv <- read_csv("hello.csv")

# Op: op_002_sort
ds_001_sorted <- source_hello.csv %>% arrange(id)

# Op: op_003_save
write_csv(ds_001_sorted, "hello_sorted.csv")
```

05_target_verification.txt (R run)

```
Command: Rscript dist/pipeline.R
Status: ✅ Success

=== OUTPUT ===

── Column specification ────────────────────────────────────────────────────────
Delimiter: ","
chr (1): name
dbl (1): id
```

Supported Features

```
Data Loading
- GET DATA /TYPE=TXT /FILE='filename.csv' /DELIMITERS=','
  - Loads CSV files with comma delimiters - Generates `read_csv()` in R

- GET DATA /TYPE=SAV /FILE='filename.sav'
  - Loads SPSS .sav files - Generates `read_sav()` in R

Data Manipulation
- SORT CASES BY variable(s) (A/D)
- COMPUTE target = expression
  - Basic arithmetic and logical expressions
  - Variable assignments

- IF (condition) target = expression
  - Generates `mutate()` with `if_else()` in R

- RECODE source_vars (...) INTO target_vars
  - Variable recoding with mapping rules
```

Cont

```
Data Filtering
- SELECT IF (condition)
  - Row filtering based on conditions
  - Generates `filter()` in R

Data Joining
- MATCH FILES /FILE=* /TABLE='file' /BY key
  - Inner and left joins
  - Multiple key variables
  - Generates `inner_join()` and `left_join()` in R

Aggregation
- AGGREGATE /BREAK=group_vars /target=func(source)
  - Grouped aggregations
  - Functions: MEAN, SUM, MAX, MIN
  - Generates `group_by()` and `summarise()` in R

Schema Definition
- DATA LIST FREE / var1 (F8.0) var2 (A10)
  - Variable type definitions
  - Basic format specifications
  - Injected into IR schema

Data Saving
- SAVE OUTFILE='filename.sav'
  - Saves to SPSS .sav format
  - Generates `write_sav()` in R

- SAVE OUTFILE='filename.csv'
  - Saves to CSV format
  - Generates `write_csv()` in R
```

Supported Functions and Operators

```
Mathematical Functions
- TRUNC() → floor()
- RND() → round()
- ABS() → abs()
- MOD(a,b) → a %% b

String Functions
- CONCAT() → paste0()
- PASTE() → paste0()
- STR_C() → str_c()

Statistical Functions
- MEAN() → mean()
- LAG() → lag()

Logical Operators
- AND → &
- OR → |
- <> → !=
- = → == (in expressions)

Conditional Functions
- IF_ELSE() → if_else()
- CASE_WHEN() → case_when()
- NA_IF() → na_if()
```

Cont

```
Date Functions
- DATE.MDY(m,d,y) → make_date(year=y, month=m, day=d)

Special Values
- $SYSMIS → NA

Partially Supported Features

Conditional Logic
- DO IF / END IF blocks
  - Basic structure recognized
  - May have formatting inconsistencies in generated conditions

Generic Commands
- Unknown SPSS commands parsed as generic nodes
- Logged as warnings in generated R code
- No functional implementation
```

Unsupported Features

```
Data Input
- Complex DATA LIST formats beyond FREE
- BEGIN DATA / END DATA inline data blocks (skipped)
- Other file types beyond TXT and SAV
- Complex delimiter specifications

Advanced Functions
- Most SPSS-specific functions not in the registry
- Advanced string manipulation beyond CONCAT
- Complex date/time operations beyond MDY
- Missing value patterns beyond $SYSMIS

Control Flow
- Complex nested IF/DO IF structures
- LOOP/END LOOP constructs
- Macro definitions and calls

Output
- Complex SAVE options and formats
- EXPORT commands
- PRINT/WRITE statements

Statistical Procedures
- All statistical analysis commands (DESCRIPTIVES, FREQUENCIES, etc.)
- Parsed as generic nodes with warnings

File Management
- FILE HANDLE definitions
- Complex file path handling
- Temporary file management
```

WAS RADAR

Speaker notes